

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Artificial Intelligence Application Development and Jira
AI SELF STUDY

Roll No.: T005	Name: Ritesh Chaubey
Class: TYIT	Batch: 1
Date of Assignment: 29-07-2026	Date/Time of Submission: 29-07-2026

1. Design PEAS description for an Online Food Delivery Agent and classify its environment.

Code:-

```
table = {
    ('Yes', 'Yes', 'No'): 'Accept Order',
    ('Yes', 'No', 'No'): 'Wait for Restaurant',
    ('Yes', 'Yes', 'Yes'): 'Deliver Order',
    ('No', 'Yes', 'No'): 'Wait for Order',
    ('No', 'No', 'No'): 'Stay Idle',
    ('No', 'Yes', 'Yes'): 'Return to Restaurant',
    ('Yes', 'No', 'Yes'): 'Pick Up Order',
    ('No', 'No', 'Yes'): 'Go Offline'
}

percepts = []

def food_delivery_agent(percept):
    percepts.append(percept)
    action = table.get(percept, "Wait")
    return action

print("Online Food Delivery Table-Driven Agent")

n = int(input("Enter number of percepts: "))

for i in range(n):
    print(f"\nPercept {i+1}")

    order = input("Order Available? (Yes/No): ").capitalize()
    restaurant = input("Restaurant Ready? (Yes/No): ").capitalize()
    delivery = input("Delivery Assigned? (Yes/No): ").capitalize()

    percept = (order, restaurant, delivery)

    action = food_delivery_agent(percept)

    print("Current Percept:", percept)
    print("Agent Action:", action)
```

```
print("Ritesh Chaubey T005")
```

Output:-

```
Online Food Delivery Table-Driven Agent

Percept 1
Current Percept: ('Yes', 'Yes', 'Yes')
Agent Action: Deliver Order

Percept 2
Current Percept: ('No', 'Yes', 'Yes')
Agent Action: Return to Restaurant

Percept 3
Current Percept: ('No', 'No', 'No')
Agent Action: Stay Idle
Ritesh Chaubey T005
```

2. Implement Breadth-First Search to find the shortest path in a simple maze.

Code:-

```
from collections import deque
```

```
maze = [
    ['S', '.', '.', '#'],
    ['#', '.', '#', '.'],
    ['.', '.', '.', '.'],
    ['#', '#', '.', 'G']
]
```

```
rows, cols = len(maze), len(maze[0])
```

```
# Find Start and Goal
```

```
for i in range(rows):
    for j in range(cols):
        if maze[i][j] == 'S':
            start = (i, j)
        if maze[i][j] == 'G':
            goal = (i, j)
```

```
queue = deque([(start, [start])])
visited = {start}
```

```
while queue:
```

```
    (x, y), path = queue.popleft()
```

```
    if (x, y) == goal:
        print("Shortest Path:", path)
        break
```

```
    for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        nx, ny = x + dx, y + dy
```

```

    if (0 <= nx < rows and
        0 <= ny < cols and
        maze[nx][ny] != '#' and
        (nx, ny) not in visited):

        visited.add((nx, ny))
        queue.append(((nx, ny), path + [(nx, ny)]))

```

```
print("Ritesh Chaubey T005")
```

Output:-

```

Shortest Path: [(0, 0), (0, 1), (1, 1), (2, 1), (2, 2), (3, 2), (3, 3)]
Ritesh Chaubey T005

```

3. Apply Bayes' Rule to calculate the probability of rain given cloudy weather.

Code:-

```

P_rain = 0.3
P_cloudy_given_rain = 0.8
P_cloudy = 0.5
P_rain_given_cloudy = (P_cloudy_given_rain * P_rain) / P_cloudy
print("Probability of Rain given Cloudy Weather =", round(P_rain_given_cloudy, 2))
print("Ritesh Chaubey T005")

```

Output:-

```

Probability of Rain given Cloudy Weather = 0.48
Ritesh Chaubey T005

```

4. Solve the Missionaries and Cannibals problem using BFS or DFS.

Code:-

```

from collections import deque

start = (3, 3, 1)
goal = (0, 0, 0)

def valid(s):
    m, c, _ = s
    return (m == 0 or m >= c) and (3 - m == 0 or 3 - m >= 3 - c)

moves = [(1,0), (2,0), (0,1), (0,2), (1,1)]

queue = deque([(start, [start])])
visited = {start}

while queue:
    (m, c, b), path = queue.popleft()

    if (m, c, b) == goal:
        print("Solution Path:")

```

```

    for p in path:
        print(p)
    break

    for dm, dc in moves:
        if b == 1:
            ns = (m - dm, c - dc, 0)
        else:
            ns = (m + dm, c + dc, 1)

        if (0 <= ns[0] <= 3 and
            0 <= ns[1] <= 3 and
            valid(ns) and
            ns not in visited):

            visited.add(ns)
            queue.append((ns, path + [ns]))

```

```
print("Ritesh Chaubey T005")
```

Output:-

```

Solution Path:
(3, 3, 1)
(3, 1, 0)
(3, 2, 1)
(3, 0, 0)
(3, 1, 1)
(1, 1, 0)
(2, 2, 1)
(0, 2, 0)
(0, 3, 1)
(0, 1, 0)
(1, 1, 1)
(0, 0, 0)
Ritesh Chaubey T005

```

5. Implement A* Search for a graph and compare its result with UCS.

Code:-

```
import heapq
```

```

graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 2), ('E', 5)],
    'C': [('F', 1)],
    'D': [('G', 5)],
    'E': [('G', 1)],
    'F': [('G', 3)],
    'G': []
}

```

```
}
```

```
heuristic = {  
    'A': 7,  
    'B': 6,  
    'C': 2,  
    'D': 3,  
    'E': 1,  
    'F': 1,  
    'G': 0  
}
```

```
def astar(start, goal):  
    pq = [(heuristic[start], 0, start, [start])]  
    visited = set()  
  
    while pq:  
        f, cost, node, path = heapq.heappop(pq)  
  
        if node == goal:  
            return path, cost  
  
        if node in visited:  
            continue  
  
        visited.add(node)  
  
        for neighbor, edge_cost in graph[node]:  
            heapq.heappush(  
                pq,  
                (cost + edge_cost + heuristic[neighbor],  
                 cost + edge_cost,  
                 neighbor,  
                 path + [neighbor])  
            )
```

```
def ucs(start, goal):  
    pq = [(0, start, [start])]  
    visited = set()  
  
    while pq:  
        cost, node, path = heapq.heappop(pq)  
  
        if node == goal:  
            return path, cost  
  
        if node in visited:
```

```

        continue

    visited.add(node)

    for neighbor, edge_cost in graph[node]:
        heapq.heappush(
            pq,
            (cost + edge_cost,
             neighbor,
             path + [neighbor])
        )

    astar_path, astar_cost = astar('A', 'G')
    ucs_path, ucs_cost = ucs('A', 'G')

    print("A* Search")
    print("Path:", astar_path)
    print("Cost:", astar_cost)

    print("\nUniform Cost Search")
    print("Path:", ucs_path)
    print("Cost:", ucs_cost)

    print("\nComparison")
    if astar_cost == ucs_cost:
        print("Both algorithms found the same optimal cost.")
    else:
        print("A* and UCS found different costs.")

    print("Ritesh Chaubey T005")
    Output:-

```

```

A* Search
Path: ['A', 'B', 'E', 'G']
Cost: 7

Uniform Cost Search
Path: ['A', 'B', 'E', 'G']
Cost: 7

Comparison
Both algorithms found the same optimal cost.
Ritesh Chaubey T005

```

